

Filtering audit: what FreeBayes filters, and what the extraction keeps

panFreebayes · answer to Pjotr’s Milestone-1 question

Contents

1	The question and the method	1
2	Answer	1
3	Complete filter inventory (all present in the extraction)	2
3.1	Read-level — <code>updateAlignmentQueue</code> (<code>AlleleParser.cpp:1954</code>)	2
3.2	Read-level — <code>--limit-coverage</code> random downsample ★	2
3.3	Base / allele-level — <code>registerAlignment</code> , <code>makeAllele</code> , <code>getAlleles</code>	2
3.4	Candidate-allele-level — <code>genotypeAlleles</code> (<code>AlleleParser.cpp:3843</code>) + main loop	3
3.5	Site / output-level — <code>freebayes.cpp</code>	3
4	Driver-only code, and whether it filters	3
4.1	The one partial gap: <code>-t</code> BED as an exclusion mechanism	5
5	<code>freebayes-parallel</code> and post-hoc filtering	5
6	Recommendation	6

1 The question and the method

Pjotr’s question: *“Is there any existing filtering in FreeBayes that isn’t included in what we extracted? Anything that filters candidate variants or reads that we may have implicitly dropped by scoping to ‘engine only’?”*

Method: every `if (...) { skip = true }, continue, pop_front, erase, remove_if`, threshold comparison and `--flag-gated` branch in the `read` → `candidate-allele` → `emitted-record` path was enumerated from `freebayes.cpp`, `AlleleParser.cpp`, `Parameters.cpp`, `Allele.cpp`, `Sample.cpp` and `BedReader.cpp`, and each was classified as **in the extraction** or **driver-only**, cross-checked against the Milestone-0 “driver vs engine” split and the actual extraction (`src/panfreebayes/panfreebayes_core.cpp`).

The relevant fact about the extraction: `callVariantsArgv` is a **verbatim** copy of `freebayes.cpp`’s `main()` loop body (the only diffs are `argv` marshalling, the output stream, and deleted comments — confirmed by `diff`), and it drives an **unmodified** `AlleleParser`. So the question reduces to: *what filtering only runs on a code path that panfreebayes cannot reach?*

2 Answer

No candidate-variant or read filter is lost. Every read-level, base-level, allele-level, candidate-level and site-level filter in FreeBayes runs identically in panFreebayes, because the entire `main()` loop and the entire `AlleleParser` are included unchanged.

The only things scoped out are `-r/--region`, `-t/--targets`, `-c/--stdin` and `-L/--bam-list`. Of these, *only `-t` has any filtering character*, and:

- `-t` is **positional windowing**, not a candidate-variant filter — it makes the driver **visit** only positions inside the BED intervals (by jumping between them). `AlleleParser::inTarget()`, the one function that could apply a BED test inside the calling loop, **is never called** — it is dead code (`AlleleParser.cpp:866`, zero call sites).
- panFreebayes replaces it structurally: “one bubble = one reference FASTA + one BAM”, analysed whole. There is no sub-region to exclude because the input **is** the region.

The single caveat (§4): if a future use wants to **exclude** a sub-region of a bubble (a known-bad repeat, say), FreeBayes could do that with `-t`; panFreebayes cannot, and the work-around is to pre-mask the reference or pre-filter the BAM.

3 Complete filter inventory (all present in the extraction)

Every entry below runs in `panfreebayes` call exactly as in stock single-process `freebayes`. Defaults from `Parameters.cpp`; ★ = overridden by the panFreebayes acceptance command (`--pooled-continuous --min-alternate-count 2 --min-alternate-fraction 0.2 --limit-coverage 200`).

3.1 Read-level — `updateAlignmentQueue` (`AlleleParser.cpp:1954`)

filter	param / default	where
read-group not in analysed sample set	—	:2010
duplicate read	<code>--use-duplicate-reads off</code> $\Rightarrow$ dropped	:2016
unmapped	—	:2022
zero aligned bases	—	:2028
secondary alignment	<code>SecondaryFlag()</code>	:2034
coordinate out of order	—	:2040
mapping quality	<code>-m / MQL0 = 1</code>	:2051
per-position coverage cap (deterministic)	<code>-g / --skip-coverage = 0</code> (off)	:2075
base-quality cap	<code>--base-quality-cap = 0</code> (off)	:2069
no alleles produced	—	:2104
mismatch fraction	<code>-z / --read-max-mismatch-fraction = 1.0</code> (off)	:2105
absolute mismatch count	<code>-U / --read-mismatch-limit = 10⁷</code> (off)	:2106
SNP count	<code>--read-snp-limit = 10⁷</code> (off)	:2107
indel count	<code>--read-indel-limit = 10⁷</code> (off)	:2108

3.2 Read-level — `--limit-coverage` random downsample ★

`freebayes.cpp:163–220` (in the lifted loop). Per site, per sample: if coverage exceeds `--limit-coverage`, alleles are dropped at random (`rand()`, seeded `srand(13)` at :109) until the cap is met. In the extraction. **Non-deterministic across differently-chunked runs** — see the realignment doc §7 and the M0 report §7. panFreebayes re-seeds per call and never chunks, so it reproduces single-process FreeBayes.

3.3 Base / allele-level — `registerAlignment`, `makeAllele`, `getAlleles`

filter	param / default	where
mismatch quality to count as a mismatch	<code>-Q / BQL2 = 10</code>	AlleleParser.
read base not ACGT $\Rightarrow$ ALLELE_NULL (uncallable)	—	:1465, :1515
reference base N $\Rightarrow$ position skipped	—	freebayes.cpp
terminal deletion (first/last CIGAR op) ignored	—	AlleleParser.
soft clip $\Rightarrow$ ALLELE_NULL	—	:1783
<code>--trim-complex-tail</code> tail split	0 (off)	:1840

haplotype-basis-allele gating $\Rightarrow$ demote to reference	--haplotype-basis-alleles (off)	:1222
allele quality	-q / BQL0 = 0	getAlleles, :3
allele currentBase == "N" or empty alt	—	:3739
allele-type mask	--no-snps / --no-indels / --no-mnps / --no-complex (all on)	freebayes.cpp

3.4 Candidate-allele-level — `genotypeAlleles` (`AlleleParser.cpp:3843`) + main loop

filter	param / default	where
group alt not all-ACGT	—	:3860
group size	-G / --min-alternate-total / minAltTotal = 1	:3864
supporting base-quality sum	-3 params / minSupportingAlleleQualitySum	:3877
supporting mapping-quality sum	minSupportingMappingQualitySum	:3877
per-sample alt quality sum	--min-alternate-qsum / minAltQSum = 0	:3927
per-sample alt count ★	-C / --min-alternate-count = 2	:3928
per-sample alt fraction ★	-F / --min-alternate-fraction = 0.05	:3929
keep only N best alleles	-n / --use-best-n-alleles = 0 (all)	:3949+
sufficientAlternateObservations pre-gate	uses -C and -F	freebayes.cpp:227
--min-coverage	-! / minCoverage = 0	freebayes.cpp:157
coverage == 0 $\Rightarrow$ skip	—	freebayes.cpp:154
only the reference allele survives $\Rightarrow$ skip	—	freebayes.cpp:312

3.5 Site / output-level — `freebayes.cpp`

filter	param / default	where
posterior probability of variation	-P / --pvar / PVL = 0.0	:645
emit only if alts non-empty (unless PVL == 0)	—	:645
--report-monomorphic	off	:226, :231
--report-genotype-likelihood-max	off	:603
no sample data likelihoods $\Rightarrow$ skip	—	:393

With --pooled-continuous and the default PVL = 0.0, the site-level gate reduces to “emit if there is at least one alt allele that cleared the candidate-level filters” (`freebayes.cpp:574–582, :645`).

4 Driver-only code, and whether it filters

driver feature	what gap? it does
-r / --region	Analysis positional window; panFreebayes uses [0, contigLen) only chr:S- E;

	loadTargets (:722), win- dow clip in toNextPosition (:2901)
-t / --targets (BED)	Partial — see below only po- si- tions in- side BED in- ter- vals, by jump- ing be- tween them (toNextTarget); in- ter- val tree built at :792
AlleleParser::inTarget()	Dead code — 0 call sites (:866) test “cur- rent po- si- tion in- side a BED tar- get”
BAM-index region seek (SetRegion)	Not Target (:2732) — only

- `vcffirstheader` — keeps one header;
- `vcfstreamsort` — re-sorts records within a 1000-record window (chunk boundaries interleave);
- `vcfuniq` — removes records that are **byte-identical to the previous line**, which only happens for a variant reported by two adjacent chunks that both overlap it.

A single-region panFreeBayes run produces no chunk-boundary duplicates, so there is nothing for `vcfuniq` to remove. The realignment doc §7 / M0 §7 point stands separately: chunked and un-chunked runs can **genuinely** disagree on `--limit-coverage` sites because of the shared PRNG — that is a divergence in **which reads survive**, upstream of any of these post-filters, and panFreeBayes matches the **single-process** result.

6 Recommendation

1. Record in the milestone report that the extraction preserves 100% of FreeBayes' read/base/allele/candidate/site filtering, and that the only scoped-out capability is `-r/-t` positional restriction, intentionally replaced by the per-bubble reference model.
2. If sub-bubble exclusion is ever needed, do it by masking the local reference (N) or pre-filtering the local BAM — not by adding region flags back to the core (which would re-introduce the BAM-index dependency the guard exists to prevent).
3. `AlleleParser::inTarget()` is dead code in upstream FreeBayes; worth a one-line note if the maintainer contact happens (alongside the two non-determinism findings).